CrowdShield

Marchés de prédiction pour anticiper les risques de surpopulation

et financer la prévention sur Tezos

Team Project – Mineure Blockchain

Yahya AHACHIM
Corentin COLMEL
Nassim BERKHLI
Theodore VAUNOIS
Etienne MARTIN

10 février 2026

Table des matières

1 Présentation du projet **2**
1.1 Titre . . . 2
1.2 Idée générale (use case) . . . 2
1.3 Pourquoi une blockchain publique est utile . . . 2
1.4 Problème clé : qui décide du résultat ? . . . 2
1.5 Fonctionnalités principales . . . 2
1.6 Contrats . . . 3

2 Plan détaillé (spécification technique) **3**
2.1 Architecture d'ensemble (on-chain / off-chain) . . . 3
2.1.1 Composants . . . 3
2.1.2 Flux global . . . 3
2.2 Diagrammes . . . 3
2.3 Maquettes UI . . . 3
2.3.1 Diagramme interactions on-chain / off-chain . . . 6

3 Spécification : Contrat A `CrowdShieldMarket` **6**
3.1 Storage (structure proposée) . . . 6
3.2 Types . . . 7
3.3 Entrypoints . . . 7
3.3.1 `create_market(question, location, threshold, end_time)` . . . 7
3.3.2 `bet(market_id, side)` . . . 7
3.3.3 `start_resolving(market_id)` . . . 7
3.3.4 `receive_outcome(market_id, result)` (callback oracle) . . . 7
3.3.5 `claim(market_id)` . . . 7
3.4 Cas particulier : aucun gagnant . . . 7

4 Spécification : Contrat B `OracleCommitReveal` **8**
4.1 Objectif . . . 8
4.2 Storage (structure proposée) . . . 8
4.3 Entrypoints . . . 8
4.3.1 `register_reporter()` (payable) . . . 8
4.3.2 `commit(market_id, commit_hash)` . . . 8
4.3.3 `reveal(market_id, result, salt)` . . . 8
4.3.4 `finalize(market_id)` . . . 8
4.4 Règles d'incitation (exemple) . . . 8

5 Contrat C `PreventionFund` (optionnel) **9**
5.1 But . . . 9
5.2 Entrypoints . . . 9

6 Plan de tests SmartPy **9**
6.1 Tests du Market . . . 9
6.2 Tests de l'Oracle . . . 9
6.3 Tests d'intégration . . . 9

1 Présentation du projet

1.1 Titre

CrowdShield — Marchés de prédiction pour anticiper les risques de surpopulation et financer la prévention, avec résolution décentralisée (oracle commit–reveal).

1.2 Idée générale (use case)

CrowdShield est une dApp de marchés de prédiction où une communauté peut miser sur un résultat binaire de type **OVER/UNDER** lié à une situation de foule. L'objectif est d'anticiper des épisodes de surpopulation lors d'événements publics en proposant des questions de type :

Le nombre de personnes sur la place X entre 18h et 20h sera-t-il supérieur à 15 000 ?

Les frais prélevés sur les marchés alimentent automatiquement un **fonds de prévention** destiné à financer des mesures de sécurité (signalétique, barriérage, renfort de médiation, etc.). Ainsi, l'activité du marché contribue directement à des actions concrètes de prévention.

1.3 Pourquoi une blockchain publique est utile

L'utilisation de smart contracts sur une blockchain publique (Tezos) apporte des garanties clés :

- **Transparence** : les règles du marché (mises, frais, calcul des gains) sont encodées on-chain et vérifiables par tous.
- **Censure limitée** : les paiements et la logique de règlement ne dépendent pas d'un serveur central.
- **Réduction de la confiance** : la détermination du résultat n'est pas confiée à une seule entité ; elle repose sur un mécanisme d'oracle décentralisé.
- **Auditabilité** : historique on-chain des marchés, des résultats et des transferts au fonds.

1.4 Problème clé : qui décide du résultat ?

Pour éviter qu'un administrateur impose un résultat arbitraire, CrowdShield intègre un oracle **commit–reveal** avec **stake** et **slashing** :

- plusieurs reporters publient un engagement cryptographique (*commit*) sur le résultat ;
- après une période définie, ils révèlent leur vote (*reveal*) ;
- le résultat final est décidé par **majorité** et **quorum** ;
- les reporters malhonnêtes (ou absents) sont pénalisés financièrement (*slashing*).

1.5 Fonctionnalités principales

- Création d'un marché : question, lieu, seuil, deadline.
- Mises OVER/UNDER (avec `sp.amount` en tez) jusqu'à la deadline.
- Fermeture automatique/logique du marché après la deadline.
- Résolution décentralisée via oracle commit–reveal.
- Paiement des gagnants via mécanisme `claim` (évite les boucles coûteuses).
- Prélèvement d'un fee et transfert vers un fonds (contrat séparé ou adresse dédiée).
- Sécurité : rôles, contrôles d'état, anti double-claim, anti re-résolution, etc.

1.6 Contrats

- **Contrat A — CrowdShieldMarket** : gère les marchés, les mises, le calcul des gains, et les claims.
- **Contrat B — OracleCommitReveal** : gère les reporters, commit/reveal, quorum, décision finale, slashing.
- **Contrat C (optionnel) — PreventionFund** : reçoit les frais et trace les dépenses.

2 Plan détaillé (spécification technique)

2.1 Architecture d'ensemble (on-chain / off-chain)

2.1.1 Composants

On-chain (Tezos / SmartPy) :
- CrowdShieldMarket
- OracleCommitReveal
- PreventionFund (optionnel)

Off-chain (facultatif pour le rendu) :
- UI web : liste des marchés, détail d'un marché, formulaire de mise, page de claim, page fonds.
- Indexeur / backend léger : lecture d'événements on-chain pour afficher l'historique.
- Source de données : capteurs, API, observation humaine (mais la décision finale reste on-chain via l'oracle).

2.1.2 Flux global

1. Un créateur initie un marché sur **CrowdShieldMarket**.
2. Les utilisateurs misent OVER/UNDER jusqu'à la deadline.
3. Après la deadline, le Market déclenche la phase de résolution (**RESOLVING**).
4. Les reporters exécutent **commit** puis **reveal** sur **OracleCommitReveal**.
5. L'oracle finalise le résultat et appelle le Market (*callback*) pour l'enregistrer.
6. Les gagnants appellent **claim** sur le Market.
7. Les frais sont transférés vers le **PreventionFund** (si utilisé) ou conservés sur une adresse dédiée.

2.2 Diagrammes

2.3 Maquettes UI

Cette section présente les écrans principaux de la dApp CrowdShield. Chaque interaction utilisateur correspond directement à un entrypoint de smart contract.

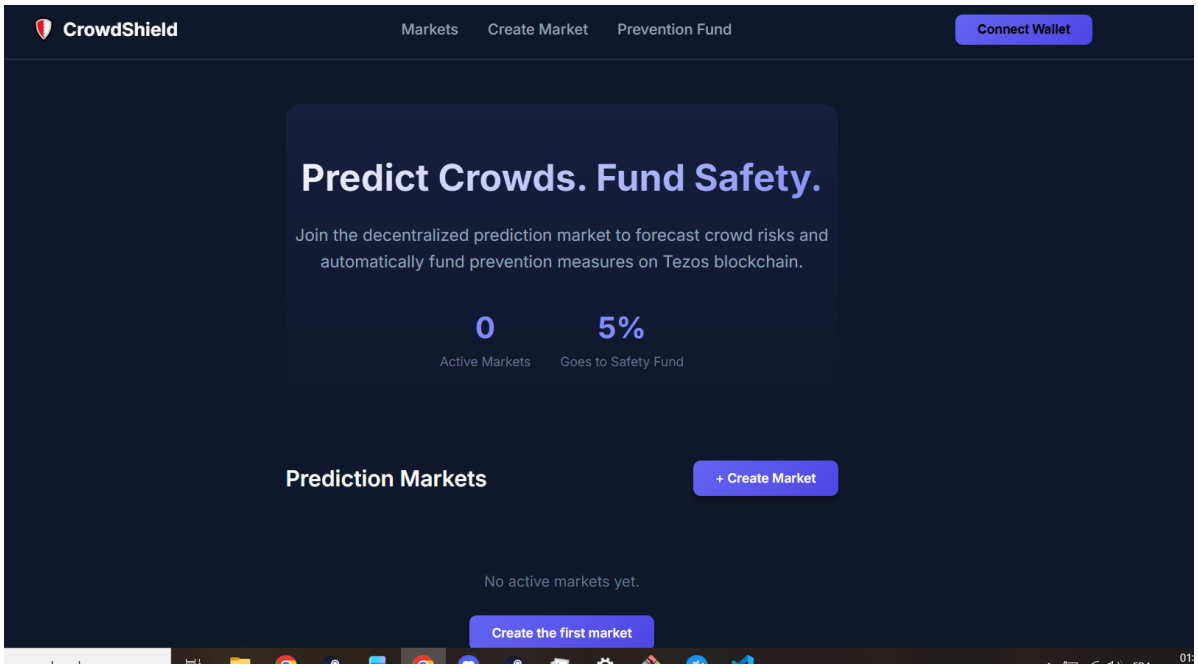

FIGURE 1 – Accueil — Liste des marchés

Affichage des marchés stockés dans CrowdShieldMarket.

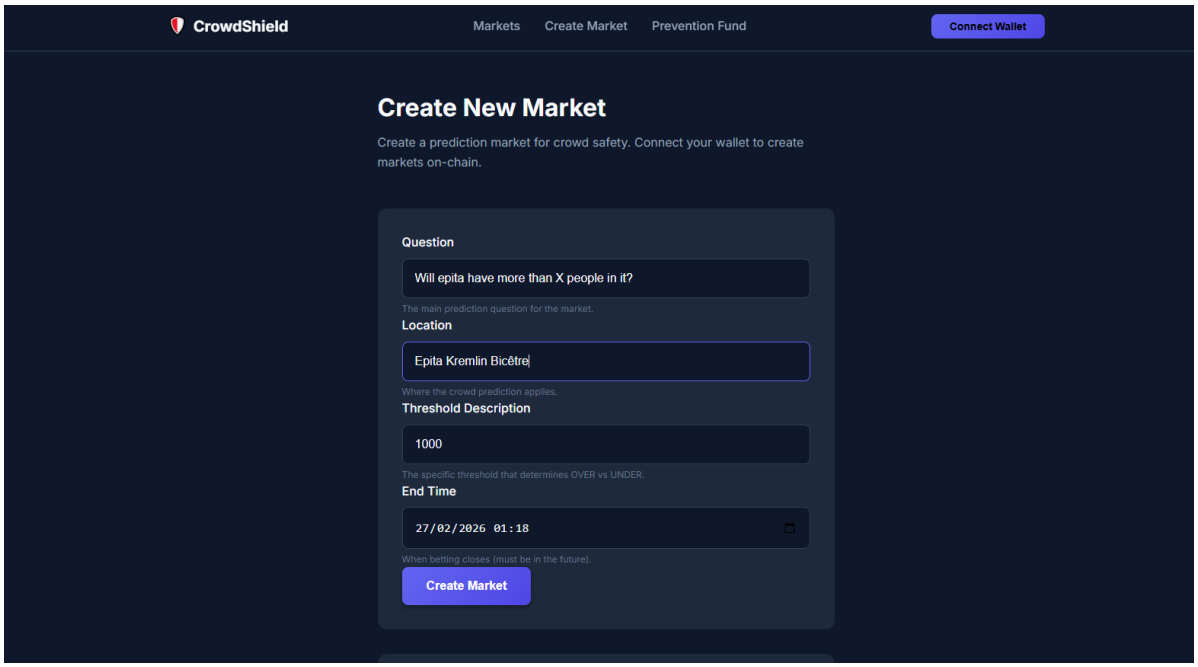

FIGURE 2 – Création d'un marché

Appel à `create_market` sur CrowdShieldMarket.

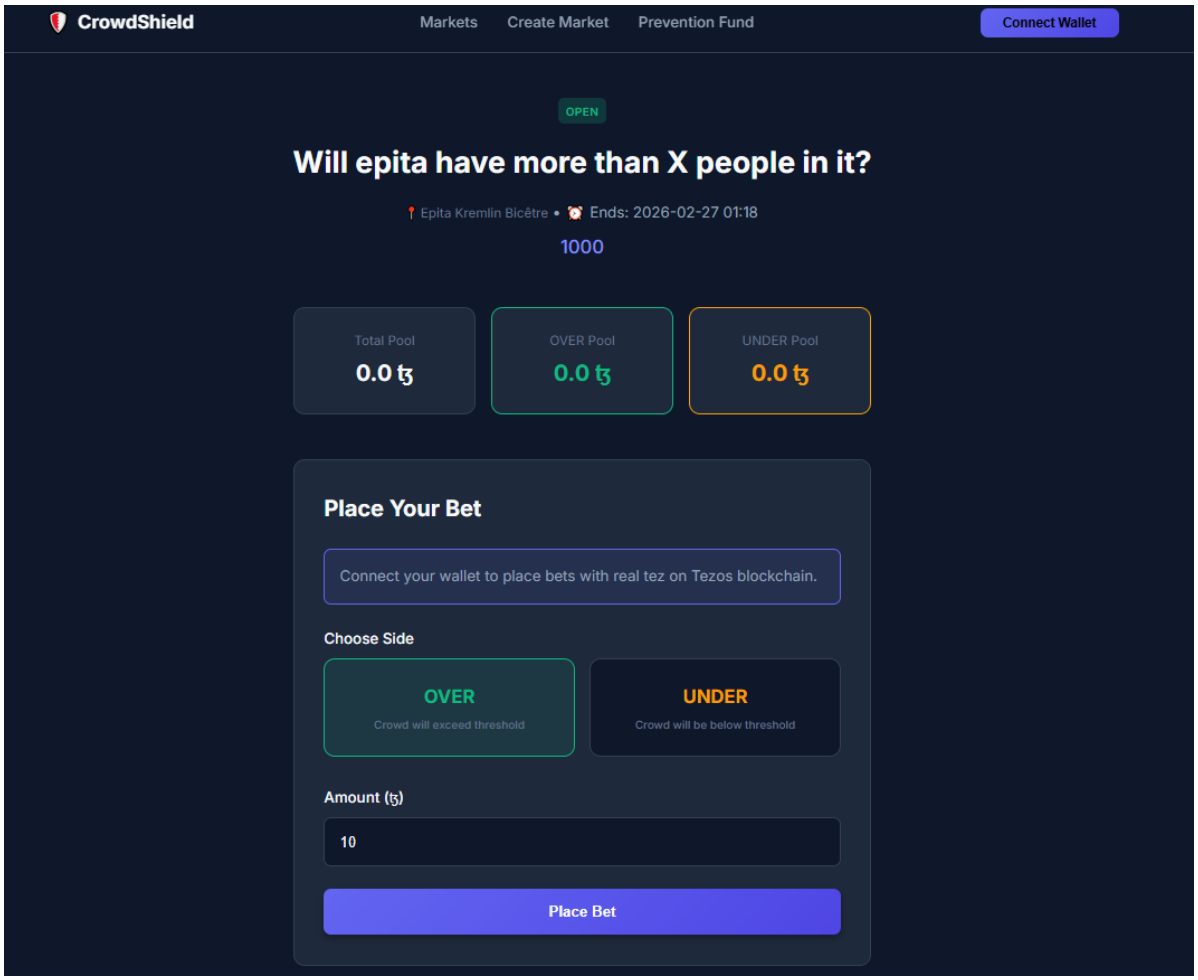

FIGURE 3 – Détail marché et mise OVER/UNDER

Appel à `bet(market_id, side)` avec `sp.amount`.

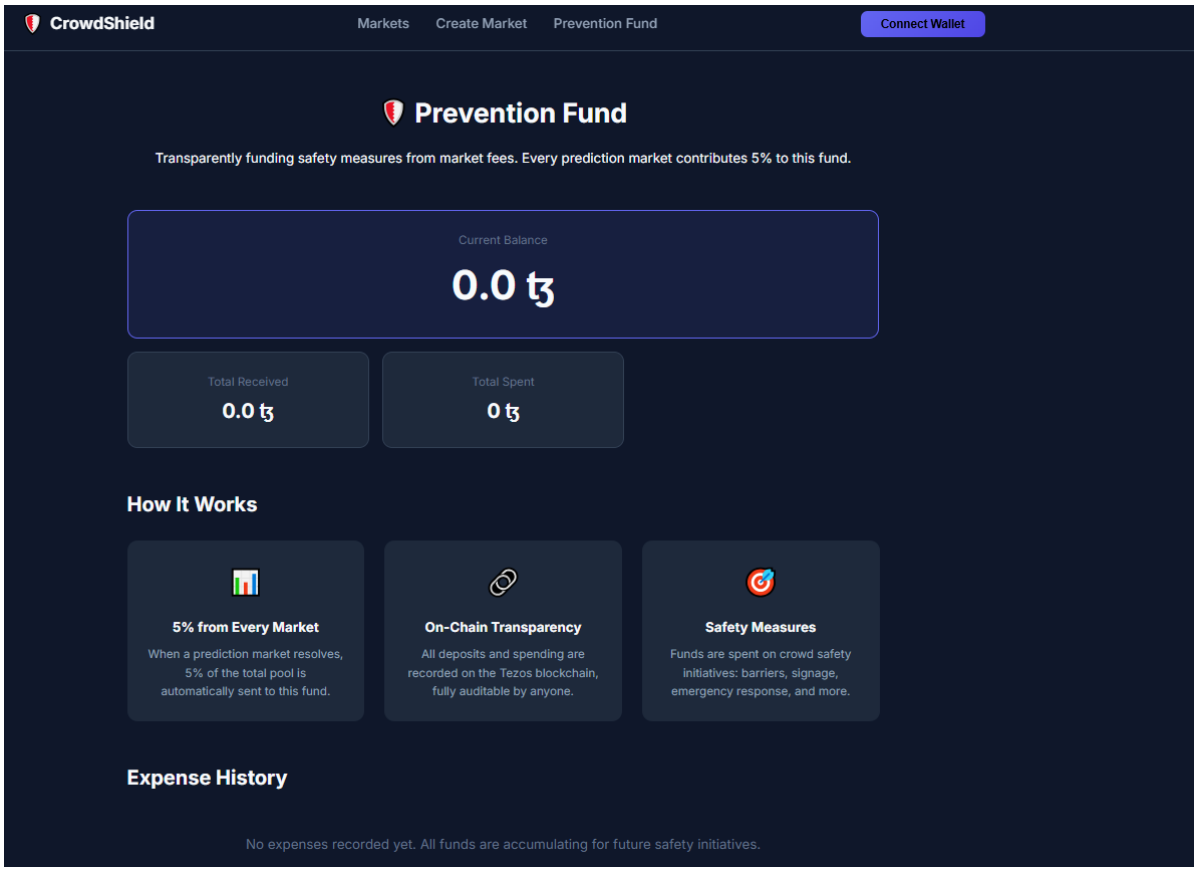

FIGURE 4 – Dashboard du fonds de prévention

2.3.1 Diagramme interactions on-chain / off-chain

```
Frontend (Flask + JS)  <---->  Wallet (Beacon / Temple)
        |
        |  appels de contrat (Taquito)
        v
+------------------- Tezos -------------------+
| CrowdShieldMarket <--> OracleCommitReveal    |
|        |                     |               |
|        +----- fees ---------> PreventionFund |
+----------------------------------------------+
```

3 Spécification : Contrat A `CrowdShieldMarket`

3.1 Storage (structure proposée)

- `admin : address`
- `oracle : address` (adresse du contrat OracleCommitReveal)
- `fee_bps : nat` (ex. 500 = 5%)
- `markets : big_map(nat, Market)`
- `next_market_id : nat`
- `bets : big_map((nat, address, Side), mutez)` (mise agrégée par utilisateur et side)
- `claimed : big_map((nat, address), bool)`

3.2 Types

- `Side` : {OVER, UNDER}
- `Status` : {OPEN, RESOLVING, RESOLVED}
- `Market` :
  - `question : string`
  - `location : string`
  - `threshold : string`
  - `end_time : timestamp`
  - `status : Status`
  - `result : option(Side)`
  - `pool_over : mutez`
  - `pool_under : mutez`

3.3 Entrypoints

3.3.1 `create_market(question, location, threshold, end_time)`

Préconditions : `end_time > now`.
Effets : ajoute un marché au statut `OPEN`.

3.3.2 `bet(market_id, side)`

Préconditions : marché `OPEN`, `now < end_time`, `sp.amount > 0`.
Effets : met à jour `pool_over/pool_under` et `bets[(market_id, sender, side)]`.

3.3.3 `start_resolving(market_id)`

Préconditions : `now >= end_time`, statut `OPEN`.
Effets : passe en `RESOLVING` et (optionnel) notifie l'oracle.

3.3.4 `receive_outcome(market_id, result)` (callback oracle)

Préconditions : `sp.sender == oracle`, statut `RESOLVING`.
Effets : fixe `result`, passe `RESOLVED`.

3.3.5 `claim(market_id)`

Préconditions : statut `RESOLVED`, pas déjà claimed.
Calcul :
- `total = pool_over + pool_under`
- `fee = total * fee_bps / 10000`
- `distributable = total - fee`
- `winning_pool = pool_result`
- `user_stake = bets[(market_id, sender, result)]`
- `payout = distributable * user_stake / winning_pool`

Effets : marque `claimed[(market_id, sender)] = True` et transfère `payout`.
Gestion du fee : transfert vers `PreventionFund.deposit()` si le contrat C est utilisé.

3.4 Cas particulier : aucun gagnant

Si `winning_pool == 0`, deux options sont possibles :

— Option 1 : tout va au fonds (incitation à participer des deux côtés).
— Option 2 : remboursement pro-rata (plus complexe).

4 Spécification : Contrat B `OracleCommitReveal`

4.1 Objectif

Permettre une résolution non arbitraire via plusieurs reporters, avec incitations économiques.

4.2 Storage (structure proposée)

— `admin : address`
— `market_contract : address` (autorisé à recevoir l'outcome)
— `min_stake : mutez`
— `reporters : big_map(address, mutez)`
— `commits : big_map((nat, address), bytes)`
— `revealed : big_map((nat, address), bool)`
— `tally : big_map(nat, record(over_votes, under_votes, reveals_count))`
— `finalized : big_map(nat, bool)`
— `reveal_deadline : big_map(nat, timestamp)`

4.3 Entrypoints

4.3.1 `register_reporter()` (payable)

Préconditions : `sp.amount >= min_stake`.
Effets : ajoute/augmente le stake du reporter.

4.3.2 `commit(market_id, commit_hash)`

Préconditions : reporter inscrit, pas déjà commit.
Effets : stocke le hash de commit.

4.3.3 `reveal(market_id, result, salt)`

Préconditions : commit existe, avant la deadline, pas déjà reveal.
Vérification : `hash(result || salt) == commit`.
Effets : incrémente le tally et marque le reveal.

4.3.4 `finalize(market_id)`

Préconditions : quorum atteint (`reveals_count >= N`) ou temps écoulé.
Effets :
— calcule le résultat (majorité OVER vs UNDER),
— applique le slashing aux absents/incohérents,
— appelle `CrowdShieldMarket.receive_outcome(market_id, result)`.

4.4 Règles d'incitation (exemple)

— Reporter absent (pas de reveal) : pénalité fixe (ex. 10% du stake).
— Reporter incohérent : pénalité plus forte (ex. 30% du stake).

— Option bonus : redistribution des pénalités aux reporters honnêtes.

5 Contrat C `PreventionFund` (optionnel)

5.1 But

Réception automatique des fees et traçabilité des dépenses.

5.2 Entrypoints

— `deposit()` (payable) : reçoit des tez.
— `spend(amount, recipient, description)` : `admin only`, journalise la dépense.

6 Plan de tests SmartPy

6.1 Tests du Market

— Création marché : champs valides, statut `OPEN`.
— Bets :
  — OK avant deadline, mise > 0.
  — Refus après deadline.
  — Refus si statut $\neq$ `OPEN`.
— Résolution :
  — `receive_outcome` refusé si `sender` $\neq$ oracle.
  — Refus si statut $\neq$ `RESOLVING`.
— Claim :
  — Gagnant reçoit le payout correct.
  — Double-claim refusé.
— Fee :
  — Calcul correct.
  — Transfert au `PreventionFund` si utilisé.

6.2 Tests de l'Oracle

— Register reporter : stake minimal, accumulation.
— Commit : reporter requis, pas de double commit.
— Reveal :
  — OK si commit match.
  — Refus si mismatch.
  — Refus hors délai.
— Finalize :
  — Quorum/majorité corrects.
  — Callback vers le Market effectué.
  — Slashing appliqué aux absents/incohérents.

6.3 Tests d'intégration

— Création marché + bets.
— Commit/reveal sur oracle.

— Finalize oracle → callback Market.
— Claims et vérification des soldes.